

面向对象程序设计C++ 第5次实验报告

一、选择题

第1题

【单选题】关于类的构造函数和析构函数，以下哪个描述是错误的？

- A. 析构函数用于清理对象占用的资源。
- B. 一个类可以有多个析构函数。
- C. 构造函数可以被重载。
- D. 析构函数在对象销毁时自动调用。

答案：B

分析：一个类只能有一个析构函数，析构函数不能重载（不接受参数、无返回值）。构造函数可以被重载（C正确），析构函数用于清理资源（A正确），析构函数在对象销毁时自动调用（D正确）。因此B是错误的描述。

第2题

【单选题】考虑下面的C++代码段，请问输出什么？

```
#include <iostream>
using namespace std;
class Box {
public:
    static int objectCount;
    Box() { ++objectCount; }
    ~Box() { --objectCount; }
};
int Box::objectCount = 0;
int main() {
    Box box1;
    Box* box2 = new Box;
    cout << Box::objectCount << endl;
    delete box2;
    cout << Box::objectCount << endl;
```

```
    return 0;
}
```

答案：B（输出 2 1）

分析：box1 构造时 objectCount 变为1，new Box 构造时变为2，此时第一次输出2。delete box2 调用析构函数，objectCount 变回1，第二次输出1。box1 是栈对象，在 main 返回时才析构，因此第二次输出时仍为1。

第3题

【多选题】在C++中使用new和delete操作符时，下列哪项描述正确？

答案：A、D

分析：

- A正确：使用 new 分配的内存必须用 delete 释放，否则会内存泄漏。
 - B错误：使用 new[] 创建的数组必须用 delete[] 释放，用 delete 会导致未定义行为。
 - C错误：释放未通过 new 分配的内存是未定义行为，非常危险。
 - D正确：new 和 delete 是C++中管理动态内存的基本操作符。
-

第4题

【单选题】关于析构函数和友元，阅读代码并选出正确的选项。

答案：A

分析：friend class School 写在 Student 类中，意味着 School 是 Student 的友元——School 可以访问 Student 的所有私有成员（包括私有析构函数），所以 School::fireStudent() 中 delete student 合法。但友元关系是单向的，Student 并非 School 的友元，因此 Student::interfereSchool() 中调用 school→access() 访问了 School 的私有成员函数 access()，会导致编译错误。删除该调用即可通过编译。

第5题

【单选题】考虑以下C++类定义，哪个描述是正确的？

答案：B（友元函数 showA 可以访问 MyClass 的私有成员 a）

分析：

- A错误：showA 是友元函数，不是成员函数。
 - B正确：showA 被声明为 MyClass 的友元，可以访问私有成员 a。
 - C错误：incrementAndShowA 是普通函数，不能直接访问私有成员，它通过公有成员函数 increment() 间接修改 a。
 - D错误：incrementAndShowA 调用的是公有成员函数和友元函数，不会编译错误。
-

第6题

【单选题】关于运算符重载的描述中，哪一个是正确的？

答案：B（运算符重载函数 operator+ 可以作为类的非成员函数实现）

分析：

- A错误：operator+ 返回的是 Complex 值对象，不是指针。
 - B正确：运算符重载既可以作为成员函数，也可以作为非成员函数实现。
 - C错误：`c1 + c2` 调用的是重载的 operator+，不是默认构造函数。
 - D错误：重载的 operator+ 保持了加法语义，没有改变原有语义。
-

第7题

【多选题】静态成员函数的特点是什么？

答案：B、C、D

分析：

- A错误：静态成员函数没有 this 指针，不能直接访问非静态成员变量。
 - B正确：可以通过 `类名::函数名()` 直接调用。
 - C正确：静态成员函数没有 this 指针，不能声明为 const（const 修饰的是 this）。
 - D正确：静态成员函数只能直接访问类的静态成员变量。
-

第8题

【单选题】考虑以下C++类定义，选择正确的选项。

```
class Test {
public:
    static const int num = 10;
    int array[num];
    Test() {}
};
```

答案：B（静态常量成员 num 可以在类内初始化）

分析：C++允许 `static const` 整型成员在类内直接初始化，num 是编译期常量，可以用来确定数组大小，代码完全合法。

二、编程题——复数运算

1. 设计思路

本题要求实现一个 Complex 复数类，支持四则运算、取共轭、取模、流输出和用户自定义字面量。核心设计如下：

- 使用 `double Re, Im` 存储实部和虚部。
- 提供双参数构造函数 `Complex(double, double)` 和单参数类型转换构造函数 `Complex(double)`，使 `double` 可以隐式转换为 `Complex`，这样四则运算只需实现一套 `Complex` 与 `Complex` 之间的版本即可自动支持 `double` 混合运算。
- 四则运算以非成员函数形式实现，按值传参，利用隐式类型转换。
- `operator~` 返回共轭复数（虚部取反），`operator-`（单目）返回取负。
- `operator<<` 需要处理实部/虚部为零的多种特殊情况，判断阈值为 `1e-5`。
- 用户自定义字面量 `operator""i` 使得可以直接用 `3i` 或 `3.0i` 表示虚数，需要 C++14 标准支持。

2. 关键代码

Complex.cpp:

```
#include "Complex.h"
```

```

#include <iomanip>

Complex::Complex(double Re, double Im) : Re(Re), Im(Im) {}

Complex::Complex(double d) : Re(d), Im(0) {}

Complex Complex::operator~() const {
    return Complex(Re, -Im);
}

Complex Complex::operator-() const {
    return Complex(-Re, -Im);
}

ostream & operator<<(ostream & os, const Complex & c) {
    const double eps = 1e-5;
    os << std::fixed << std::setprecision(2);

    if (std::fabs(c.Re) < eps && std::fabs(c.Im) < eps) {
        os << 0.0;
    } else if (std::fabs(c.Re) < eps) {
        os << c.Im << "i";
    } else if (std::fabs(c.Im) < eps) {
        os << c.Re;
    } else if (c.Im < 0) {
        os << c.Re << " - " << std::fabs(c.Im) << "i";
    } else {
        os << c.Re << " + " << c.Im << "i";
    }

    return os;
}

Complex operator+(Complex c1, Complex c2) {
    return Complex(c1.Re + c2.Re, c1.Im + c2.Im);
}

Complex operator-(Complex c1, Complex c2) {
    return Complex(c1.Re - c2.Re, c1.Im - c2.Im);
}

Complex operator*(Complex c1, Complex c2) {
    return Complex(c1.Re * c2.Re - c1.Im * c2.Im,
        c1.Re * c2.Im + c1.Im * c2.Re);
}

```

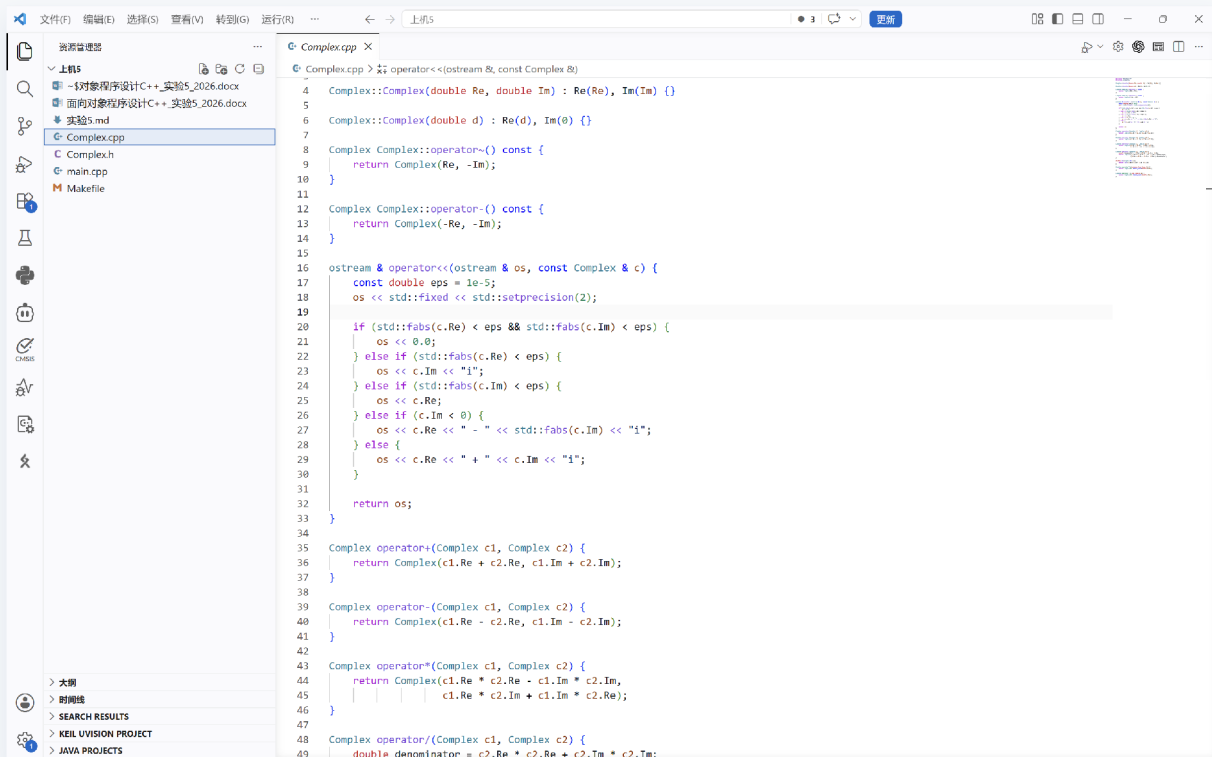
```
}
```

```
Complex operator/(Complex c1, Complex c2) {  
    double denominator = c2.Re * c2.Re + c2.Im * c2.Im;  
    return Complex((c1.Re * c2.Re + c1.Im * c2.Im) / denominator,  
        (c1.Im * c2.Re - c1.Re * c2.Im) / denominator);  
}
```

```
double abs(Complex & c) {  
    return sqrt(c.Re * c.Re + c.Im * c.Im);  
}
```

```
Complex operator""i(unsigned long long Im) {  
    return Complex(0, static_cast<double>(Im));  
}
```

```
Complex operator""i(long double Im) {  
    return Complex(0, static_cast<double>(Im));  
}
```

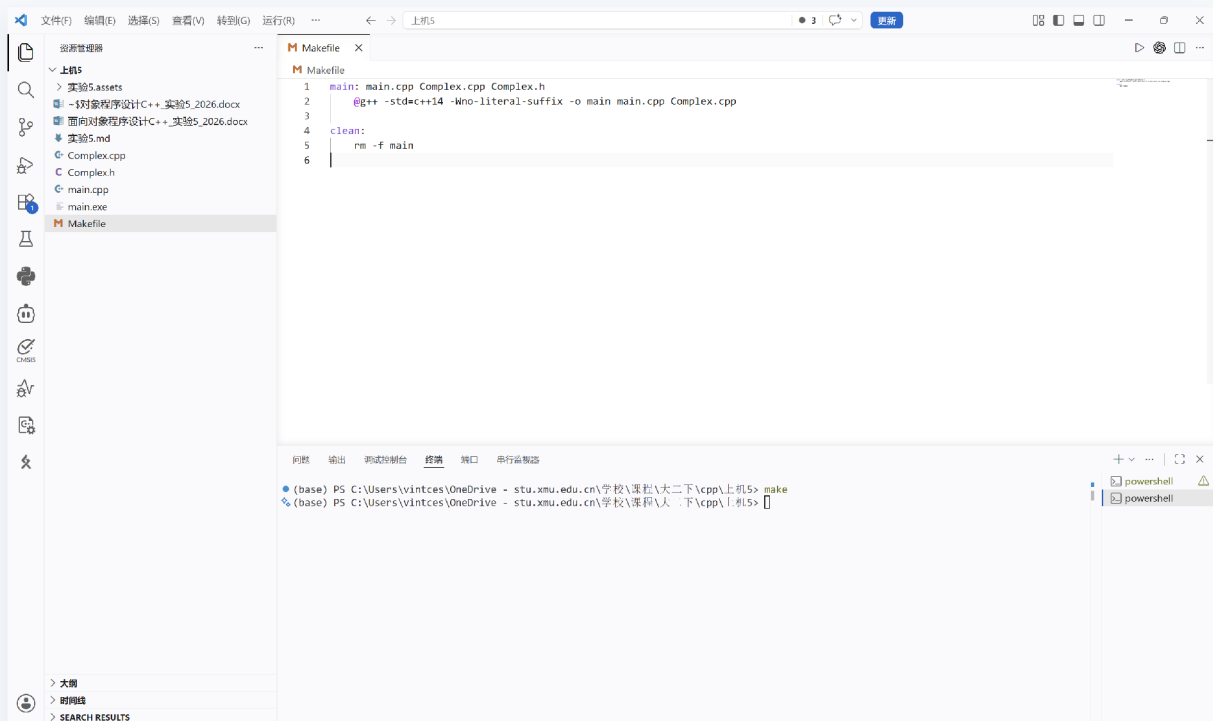


3. 编译与运行

使用 Makefile 进行编译，编译命令为 `make`：

```
main: main.cpp Complex.cpp Complex.h
    g++ -std=c++14 -o main main.cpp Complex.cpp

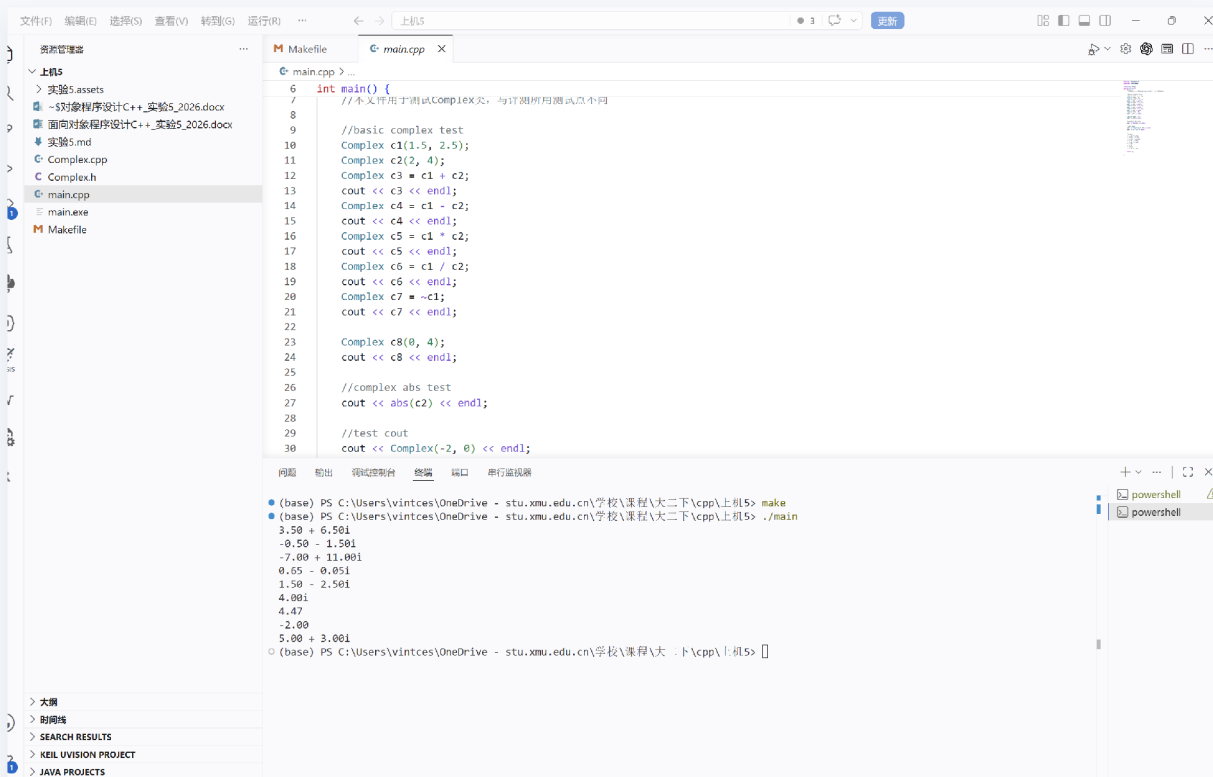
clean:
    rm -f main
```



4. 程序运行结果

执行 `./main`，输出如下：

```
3.50 + 6.50i
-0.50 - 1.50i
-7.00 + 11.00i
0.65 - 0.05i
1.50 - 2.50i
4.00i
4.47
-2.00
5.00 + 3.00i
```



```
6 int main() {
7     //本文档用于测试Complex类，与试题所用测试点不同
8
9     //basic complex test
10    Complex c1(1.5, 2.5);
11    Complex c2(2, 4);
12    Complex c3 = c1 + c2;
13    cout << c3 << endl;
14    Complex c4 = c1 - c2;
15    cout << c4 << endl;
16    Complex c5 = c1 * c2;
17    cout << c5 << endl;
18    Complex c6 = c1 / c2;
19    cout << c6 << endl;
20    Complex c7 = ~c1;
21    cout << c7 << endl;
22
23    Complex c8(0, 4);
24    cout << c8 << endl;
25
26    //complex abs test
27    cout << abs(c2) << endl;
28
29    //test cout
30    cout << Complex(-2, 0) << endl;
```

```
(base) PS C:\Users\vintces\OneDrive - stu.xmu.edu.cn\学校\课程\大二下\cpp\上机5> make
(base) PS C:\Users\vintces\OneDrive - stu.xmu.edu.cn\学校\课程\大二下\cpp\上机5> ./main
3.50 + 6.50i
-0.50 - 1.50i
-7.00 + 11.00i
0.65 - 0.05i
1.50 - 2.50i
4.00i
4.47
-2.00
5.00 + 3.00i
(base) PS C:\Users\vintces\OneDrive - stu.xmu.edu.cn\学校\课程\大二下\cpp\上机5>
```

5. 设计总结

本题的关键点：

- 1. 隐式类型转换：**通过 `Complex(double d)` 单参数构造函数，使 `double` 自动转换为 `Complex`，避免为每种 `double-Complex` 组合重复编写运算符重载。例如 `5 + 3i` 中，整数5会先通过字面量转为 `double`，再隐式转换为 `Complex(5, 0)`，然后与 `Complex(0, 3)` 相加。
- 2. 流输出的特殊情况处理：**需要判断实部和虚部是否为零（阈值 `1e-5`），分为五种情况——都为零输出 `0.00`；仅实部为零输出虚部如 `4.00i`；仅虚部为零输出实部如 `-2.00`；虚部为负输出如 `1.50 - 2.50i`；虚部为正输出如 `3.50 + 6.50i`。
- 3. 用户自定义字面量：**`operator""i` 提供了 `unsigned long long` 和 `long double` 两个重载

版本，分别处理整数和浮点数字面量，使得可以用简洁的 `3i` 或 `3.0i` 语法直接表示虚数。

4. 复数除法：使用标准公式 $(a+bi)/(c+di) = (ac+bd)/(c^2+d^2) + (bc-ad)/(c^2+d^2)i$ ，分母为 c^2+d^2 。

附录：完整源代码

Complex.h

```
#ifndef COMPLEX_H_
#define COMPLEX_H_
#include <iostream>
#include <cmath>

using std::ostream;
using std::sqrt;

class Complex {
public:
    double Re, Im;

    Complex(double Re, double Im);
    Complex(double d);
    Complex operator~() const;
    Complex operator-() const;
    friend ostream & operator<<(ostream & os, const Complex & c);
};

Complex operator+(Complex c1, Complex c2);
Complex operator-(Complex c1, Complex c2);
Complex operator*(Complex c1, Complex c2);
Complex operator/(Complex c1, Complex c2);

double abs(Complex & c);

Complex operator""i(unsigned long long Im);
Complex operator""i(long double Im);

#endif
```

Complex.cpp

```
#include "Complex.h"
#include <iomanip>

Complex::Complex(double Re, double Im) : Re(Re), Im(Im) {}

Complex::Complex(double d) : Re(d), Im(0) {}

Complex Complex::operator~() const {
    return Complex(Re, -Im);
}

Complex Complex::operator-() const {
    return Complex(-Re, -Im);
}

ostream & operator<<(ostream & os, const Complex & c) {
    const double eps = 1e-5;
    os << std::fixed << std::setprecision(2);

    if (std::fabs(c.Re) < eps && std::fabs(c.Im) < eps) {
        os << 0.0;
    } else if (std::fabs(c.Re) < eps) {
        os << c.Im << "i";
    } else if (std::fabs(c.Im) < eps) {
        os << c.Re;
    } else if (c.Im < 0) {
        os << c.Re << " - " << std::fabs(c.Im) << "i";
    } else {
        os << c.Re << " + " << c.Im << "i";
    }

    return os;
}

Complex operator+(Complex c1, Complex c2) {
    return Complex(c1.Re + c2.Re, c1.Im + c2.Im);
}

Complex operator-(Complex c1, Complex c2) {
    return Complex(c1.Re - c2.Re, c1.Im - c2.Im);
}
```

```

Complex operator*(Complex c1, Complex c2) {
    return Complex(c1.Re * c2.Re - c1.Im * c2.Im,
                   c1.Re * c2.Im + c1.Im * c2.Re);
}

Complex operator/(Complex c1, Complex c2) {
    double denominator = c2.Re * c2.Re + c2.Im * c2.Im;
    return Complex((c1.Re * c2.Re + c1.Im * c2.Im) / denominator,
                   (c1.Im * c2.Re - c1.Re * c2.Im) / denominator);
}

double abs(Complex & c) {
    return sqrt(c.Re * c.Re + c.Im * c.Im);
}

Complex operator""i(unsigned long long Im) {
    return Complex(0, static_cast<double>(Im));
}

Complex operator""i(long double Im) {
    return Complex(0, static_cast<double>(Im));
}

```

main.cpp

```

#include "Complex.h"
#include <iostream>

using std::cout;
using std::endl;
int main() {
    Complex c1(1.5, 2.5);
    Complex c2(2, 4);
    Complex c3 = c1 + c2;
    cout << c3 << endl;
    Complex c4 = c1 - c2;
    cout << c4 << endl;
    Complex c5 = c1 * c2;
    cout << c5 << endl;
    Complex c6 = c1 / c2;
    cout << c6 << endl;
    Complex c7 = ~c1;
}

```

```
cout << c7 << endl;  
Complex c8(0, 4);  
cout << c8 << endl;  
cout << abs(c2) << endl;  
cout << Complex(-2, 0) << endl;  
cout << 5 + 3i << endl;  
return 0;  
}
```